

# VitalMetrics

A Simple Health & Nutrition Calculator

*Project Report*

Submitted by: Soumyadeep Mondal

Registration No.: 26BCE11084

Course / Subject: Introduction to Problem Solving

Date: 27 Sep 2026

*Submitted as part of the VITyarthi “Build Your Own Project” flipped course evaluation.*

## 1. Introduction

Plenty of people want to keep track of their health but aren't familiar with basics such as their BMI or how many calories they burn in a day. Calculators for these numbers already exist online, but they're scattered across different sites, so I built a compact Python program called VitalMetrics that brings four common health calculations — BMI, BMR, TDEE, and Body Fat Percentage — into one place. It runs in the terminal behind a menu, letting the user run whichever calculation they need, one after another, without closing and relaunching the program each time.

## 2. Problem Statement

Most people don't have a single place to check their BMI, BMR, TDEE, and Body Fat Percentage together — each one usually has to be looked up separately. On top of that, many online calculators just return a raw number without explaining what it means, such as whether a given BMI sits in the healthy range. This project addresses both gaps with one small program that performs all four calculations and also explains the BMI result in plain terms (underweight, healthy, overweight, or obese).

## 3. Functional Requirements

The program is built around a main menu with four calculators, each implemented as its own module:

- **Module 1 – BMI Calculator:** The user supplies height (in metres) and weight (in kg). The program computes  $BMI = \text{weight} / \text{height}^2$  and reports whether the person is underweight, healthy weight, overweight, or obese based on that value.
- **Module 2 – BMR Calculator:** The user supplies gender, height (cm), weight (kg), and age. The program computes Basal Metabolic Rate using a standard equation, applying a different formula for male and female.
- **Module 3 – TDEE Calculator:** Works the same as the BMR module, but also asks for an activity level (Sedentary, Lightly Active, Moderately Active, Very Active, Extra Active); the BMR value is then multiplied by an activity-specific factor to estimate total daily calorie needs.
- **Module 4 – Body Fat Percentage Calculator:** The user selects a category (Adult male, Adult female, Boy, Girl) and enters height, weight, and age. The program estimates body fat percentage using BMI together with age.

How it works: the program displays the menu in a continuous loop. The user enters a number from 1 to 4 to run a calculator, or 0 to exit. Once a calculation finishes, the menu reappears so another option can be run if desired.

## 4. Non-Functional Requirements

- **Usability:** The menu relies on simple numbered text options, making it approachable even for non-technical users.
- **Maintainability:** The BMI and BMR formulas live in their own functions (`bmi()`, `bmr_for_male()`, `bmr_for_female()`) rather than being repeated inline, which makes future fixes or changes easier.
- **Reliability:** The menu loop keeps running until the user selects option 0 to exit; an invalid menu choice simply triggers a message and another prompt instead of crashing the program.
- **Resource usage:** The program only performs simple arithmetic on user-supplied numbers, so it runs quickly and needs very little memory.

## 5. System Architecture

The program lives in a single Python file structured in two parts: the top section holds the calculation functions (`bmi`, `bmr_for_male`, `bmr_for_female`), and the bottom section contains a loop that displays the menu, reads the

user's choice, and calls the matching function. There's no database, no file storage, and no network access — everything happens in memory while the program runs and is discarded once it closes.

| Part of Program       | What it does                                                | Where in the code                       |
|-----------------------|-------------------------------------------------------------|-----------------------------------------|
| Menu / Input-Output   | Displays the menu, reads user input, prints results         | print() and input() lines               |
| Main loop             | Determines which calculator runs based on the user's choice | while True: with if/elif                |
| Calculation functions | Perform the BMI and BMR arithmetic                          | bmi(), bmr_for_male(), bmr_for_female() |

## 6. Design Diagrams

### 6.1 Workflow Diagram (steps)

(written out step by step below — render this as an actual flowchart image before final submission)

- Start → display main menu
- User enters a choice
- If choice is 1 → run BMI calculator → display result → return to menu
- If choice is 2 → run BMR calculator → display result → return to menu
- If choice is 3 → run BMR then TDEE calculator → display result → return to menu
- If choice is 4 → run Body Fat calculator → display result → return to menu
- If choice is 0 → print a farewell message → program ends
- If choice is anything else → display “Enter a valid choice” → return to menu

### 6.2 Use Case Diagram (in words)

The user is the only actor in the system. The actions available are: Calculate BMI, Calculate BMR, Calculate TDEE, Calculate Body Fat Percentage, and Exit the program. Every calculator is reachable directly from the main menu.

### 6.3 How the Functions Connect

- Main menu calls → bmi() when the user selects option 1
- Main menu calls → bmr\_for\_male() or bmr\_for\_female() when the user selects option 2
- For TDEE (option 3), the program first calls bmr\_for\_male()/bmr\_for\_female(), then multiplies the result by an activity-level factor
- For Body Fat (option 4), the program first calls bmi(), then feeds that value into the body fat formula

Since this project doesn't use a database or persist data to a file, no ER diagram is included in this version.

## 7. Design Decisions & Rationale

- The BMI and BMR calculations were split into their own functions because each one is reused elsewhere — BMR feeds both the BMR option and the TDEE option, and BMI feeds both the BMI option and the Body Fat option. This avoids writing the same formula twice.
- A while loop drives the menu so the user can perform multiple calculations without restarting the program each time, which felt far more convenient.
- The program was kept as a command-line tool rather than given a GUI, mainly to keep the scope manageable and focus on getting the calculations right first.

## 8. Implementation Details

Language / tools used: Python 3, relying only on the built-in input()/print() functions — no external libraries required.

Formulas used in the code:

- $\text{BMI} = \text{weight (kg)} / \text{height (m)}^2$
- $\text{BMR (male)} = 10 \times \text{weight} + 6.25 \times \text{height} - 5 \times \text{age} + 5$
- $\text{BMR (female)} = 10 \times \text{weight} + 6.25 \times \text{height} - 5 \times \text{age} - 161$
- $\text{TDEE} = \text{BMR} \times \text{activity factor (1.2 to 1.9 depending on activity level)}$
- $\text{Body Fat \% (Adult male)} = 1.2 \times \text{BMI} + 0.23 \times \text{age} - 16.2$
- $\text{Body Fat \% (Adult female)} = 1.2 \times \text{BMI} + 0.23 \times \text{age} - 5.2$
- $\text{Body Fat \% (Boy)} = 1.51 \times \text{BMI} + 0.70 \times \text{age} - 2.2$
- $\text{Body Fat \% (Girl)} = 1.51 \times \text{BMI} + 0.70 \times \text{age} - 1.4$

The two core functions, taken directly from the code:

```
def bmi(a,b):  
    c=b/(a*a)  
    return(c)  
def bmr_for_male(w,h,a):  
    bmr=(10*w)+(6.25*h)-(5*a)+5  
    return(bmr)
```

How the menu works: the entire program runs inside a while True loop. It prints the menu, reads the choice the user enters, and uses if/elif statements to determine which block runs. Entering 0 breaks the loop and ends the program.

## 9. Screenshots / Results

[Add terminal screenshots here before submitting.] Useful screenshots to include: the main menu, a BMI run showing its classification, a TDEE run showing the activity-level prompt and result, and a Body Fat run.

| Module   | Sample Input                                    | Sample Output                                   |
|----------|-------------------------------------------------|-------------------------------------------------|
| BMI      | height=1.75 m, weight=70 kg                     | BMI is 22.86 kg/m <sup>2</sup> – healthy weight |
| BMR      | male, height=175 cm, weight=70 kg, age=25       | BMR is about 1673.75 kcal/day                   |
| TDEE     | same as above, activity = Moderately Active     | TDEE is about 2594.3 kcal/day                   |
| Body Fat | Adult male, height=1.75 m, weight=70 kg, age=25 | Body Fat is about 16.0 %                        |

## 10. Testing Approach

The program was tested by hand: each menu option was run with typical values, and the result was cross-checked against a separate calculator.

- Verified BMI classification using values just above and below 18.5, 24.9, and 29.9.
- Verified BMR by running identical height/weight/age figures once as male and once as female to confirm the answer changes.
- Verified TDEE by running the same profile through all five activity levels and confirming each produces a different value.
- Verified Body Fat by running each of the four categories once.

## 11. Challenges Faced

- The program doesn't validate input — entering letters instead of a number causes it to crash with an error. This wasn't handled in the current version.
- The user must type gender/category/activity level exactly as shown (e.g. “male” or “Adult male”); a slightly different entry (extra space, different capitalization) means that branch simply produces no output.
- Tracking down the correct BMR and Body Fat Percentage formulas, and making sure they were typed correctly, took some trial and error.
- Everything currently lives in one file, which made writing and testing easier but means the project isn't really split into separate modules.

## 12. Learnings & Key Takeaways

- Turning the repeated formulas (BMI, BMR) into functions instead of retyping them made the code shorter and easier to maintain.
- A while loop paired with if/elif is a straightforward way to build a menu-driven program.
- Even a simple-looking calculator needs some thought about organizing the different options (gender, activity level, category) so the correct formula is picked every time.
- Watching the program crash on bad input during testing made clear how important input validation really is.

## 13. Future Enhancements

- Add input validation (using try/except) so the program shows a helpful error message instead of crashing on bad input.
- Make the gender/category/activity inputs case-insensitive, so “Male” and “male” both work.
- Split the code into separate files (e.g. bmi.py, bmr.py, tdee.py, bodyfat.py, main.py) instead of one large file.
- Save the user's results to a file so progress can be tracked over time.
- Possibly add a simple GUI later using Tkinter instead of the terminal menu.

## 14. References

- BMI classification ranges – World Health Organization website.
- BMR formula (Mifflin-St Jeor equation) – referenced from general fitness/health websites.
- Python official documentation – docs.python.org